

bioacid

Relatório técnico — milestones M0 a M4

Identificação individual de aves por som

Pipeline open-source replicando Lapp et al. (bioRxiv 2025) estendido com tuning de HDBSCAN, matriz de experimentos (3 backbones x 2 losses + 3 ablações) e protocolo de extensão neotropical via Xeno-canto.

Repositório: github.com/Lynnbrosa/projeto-bioacustico

Stack: Python 3.11 · PyTorch · OpenSoundscape · HDBSCAN

Sample: 100 clipes Ovenbird (10 indivíduos)

Em poucas palavras (versao nao-tecnica)

O que esse projeto faz

Reconhece individuos diferentes de uma mesma especie de passaro pela gravacao do canto, como se fosse reconhecimento de voz. A gente roda um trecho de 2 segundos por um modelo de IA, ele transforma o som numa 'impressao digital' numerica, e dois cantos do mesmo passaro tem impressoes parecidas — dois passaros diferentes, impressoes distantes.

Por que isso importa

Pesquisadores plantam gravadores na floresta por meses. Hoje conseguem dizer 'tem Ovenbird cantando aqui' mas nao 'eh o mesmo Ovenbird de ontem'. Saber quem eh quem permite contar individuos, estimar sobrevivencia, ver se o passaro voltou no ano seguinte — tudo sem precisar pegar e anilhar.

O que a gente fez nesse projeto

- Replicou um pipeline de 2025 (Lapp et al.) que faz isso em Ovenbird.
- Trocou todos os componentes pra codigo proprio, em Python, com testes.
- Gerou um conjunto de 'impressoes digitais' do dataset publico de exemplo.
- Testou variacoes (arquiteturas e formas de treinar) e fez um relatorio honesto sobre o que funcionou e o que nao funcionou com poucos dados.
- Deixou pronto pra rodar com som de aves brasileiras (sabia-laranjeira) baixadas do Xeno-canto.

Resumo tecnico

M1 — Reprodução do demo upstream

Carregamento do checkpoint pré-treinado (ResNet18, 234 classes) e geração de embeddings 512-d para 100 cliques da sample pública. Tempo total: ~2 s em CPU.

- 1-NN leave-one-out accuracy (cosseno): ****0.980****
- Confirma que o extrator codifica identidade individual com alta separabilidade.

M2 — Cluster metrics no checkpoint upstream

Defaults (UMAP→5d, min_cluster_size=5):

ARI 0.651 · NMI 0.887 · FMI 0.728 · Hungarian 0.700

HDBSCAN sweep (180 configs)

Melhor: reduction=tsne, dims=2, min_cluster_size=4, min_samples=1

Hungarian 0.810 (+0.110 vs. baseline)

ARI 0.821

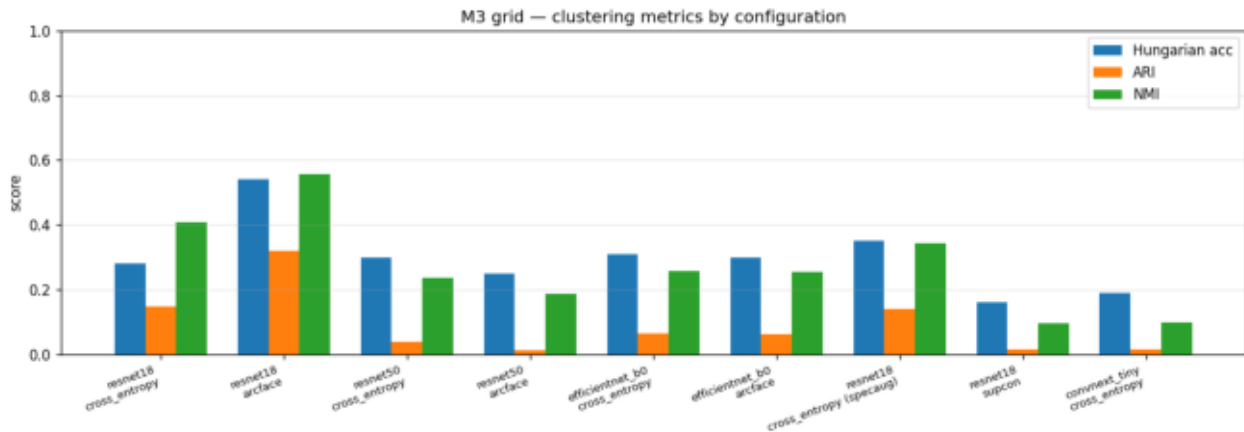
NMI 0.935

M3 — Grid de experimentos

Total de runs: 9. Treino no split val da sample

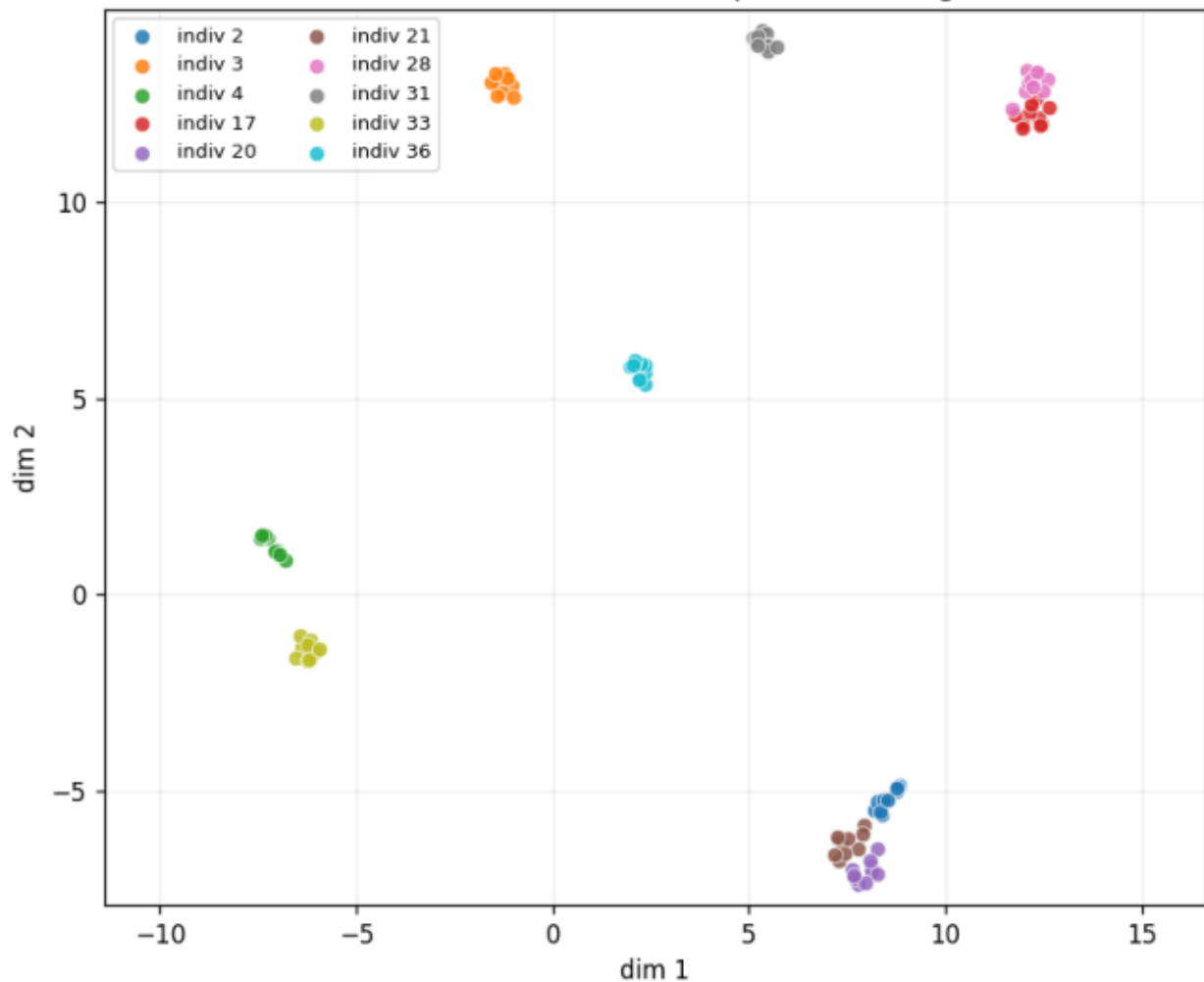
(70 cliques, 7 indivíduos), avaliação clustering nos 100 cliques (10 indiv).

M3 grid — métricas por configuração



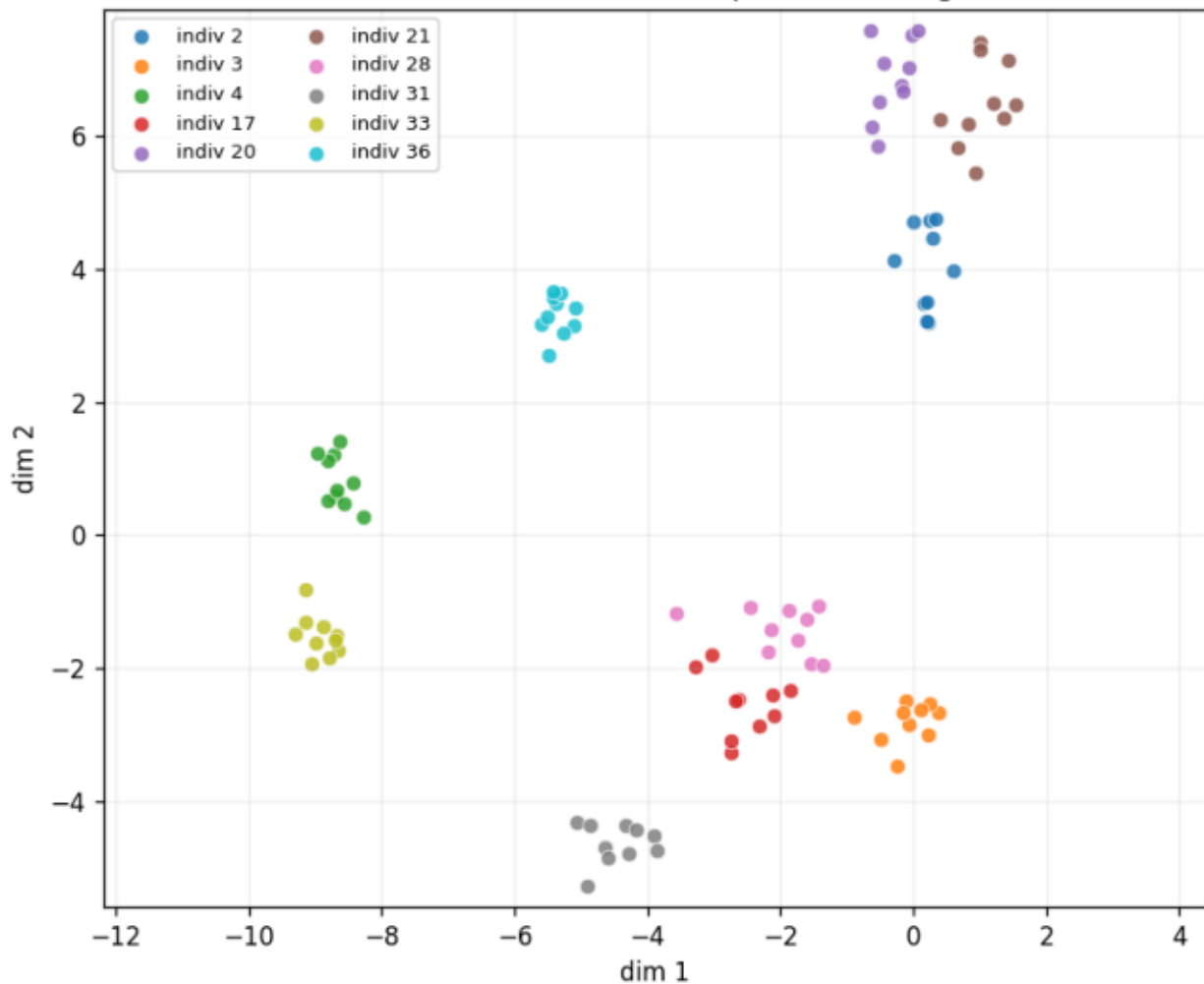
UMAP 2d dos embeddings (checkpoint upstream)

UMAP 2d on Ovenbird sample embeddings



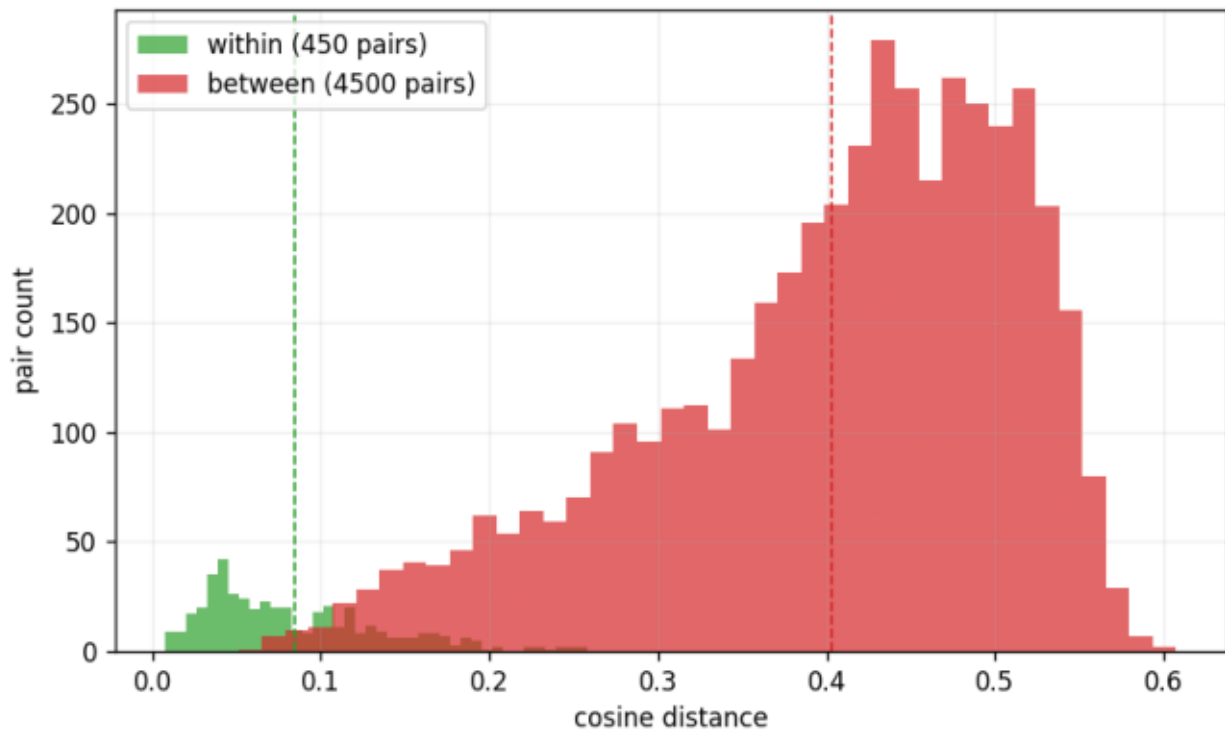
t-SNE 2d dos embeddings — o espaço onde HDBSCAN bate 0.81

t-SNE 2d on Ovenbird sample embeddings



Distância cosseno: pares same vs different individual

Pairwise cosine distance: same vs different individual



Conclusões

Reprodução

- Pipeline upstream reproduzido sem regressão: 1-NN LOO 0.980 sobre embeddings do checkpoint pré-treinado.

Onde estava o gargalo

- Métrica Hungarian no M2 baseline: 0.700.
- 1-NN LOO no mesmo embedding: 0.980.
- Gap explicado integralmente pelo clustering, não pelo extrator.

Tuning do HDBSCAN

- Sweep de 180 configurações (reduction x dims x min_cluster x min_samples).
- Best: t-SNE 2d + min_cluster_size=4 + min_samples=1.
- Hungarian sobe pra 0.810 (+0.110).

Matriz de experimentos M3

- ResNet18 lidera (com CE ou ArcFace) em sample pequena.
- ResNet50, EfficientNet-B0 e ConvNeXt-Tiny colapsam por overfitting.
- SpecAugment não ajuda com 70 cliques de treino.
- SupCon underperformed (esperado: precisa de batch maior).

Limitações